

Lesson 7: The DOM API

Intro to Programming — Code the Dream

Session Overview

By the end of this session, you will be able to:

- Explain what the DOM is and how JavaScript uses it
- Select elements on a page using `querySelector`
- Create new elements and add them to the page
- Use a loop to build a list from an array
- Get the current year dynamically using the `Date` object

Explore

What Is the DOM?

The **Document Object Model (DOM)** is a tree of every element on your page.

When your browser loads HTML, it builds this tree.

JavaScript can then read and change that tree.

The DOM is what makes web pages interactive — without reloading the page.

The DOM Tree

Your HTML becomes a tree of **nodes**.

```
document
├── <html>
│   ├── <head>
│   │   └── <title>
│   └── <body>
│       ├── <h1>
│       ├── <section>
│       └── <footer>
```

- Every tag is a **node**
- Nodes have **parents**, **children**, and **siblings**

Selecting Elements

`querySelector` finds the **first** matching element.

```
// Select by tag name
let footer = document.querySelector("footer");

// Select by ID
let skills = document.querySelector("#skills");

// Select by class
let items = document.querySelector(".card");
```

CSS selectors work here — # for ID, . for class.

querySelector vs. querySelectorAll

Method	Returns
<code>querySelector</code>	The first match (one element)
<code>querySelectorAll</code>	All matches (a list)

```
// Returns the first <h2> it finds
let heading = document.querySelector("h2");

// Returns every <h2> on the page
let allHeadings = document.querySelectorAll("h2");
```

Check for Understanding

You want to select the element with `id="skills"`. Which is correct?

- A) `document.querySelector("skills")`
- B) `document.querySelector(".skills")`
- C) `document.querySelector("#skills")`
- D) `document.querySelector("<skills>")`

Creating Elements

Three steps to add something new to the page:

```
// Step 1: Create a new element
let paragraph = document.createElement("p");

// Step 2: Set its content
paragraph.textContent = "Hello from JavaScript!";

// Step 3: Add it to the page
document.body.appendChild(paragraph);
```

No need to edit your HTML file — JavaScript does it.

innerHTML vs. textContent

Both set the content of an element — but they are different.

```
// textContent – plain text only
paragraph.textContent = "Hello!";

// innerHTML – can include HTML tags and entities
paragraph.innerHTML = "&copy; 2025 Maria Santos";
```

- Use `textContent` for plain text
- Use `innerHTML` when you need HTML (like `©`)

Getting the Current Year

Use the `Date` object — no hardcoding!

```
let today = new Date();  
let thisYear = today.getFullYear();  
  
console.log(thisYear); // e.g. 2025
```

If you hardcode 2024, your site will be wrong next year.

Check for Understanding

What does `document.createElement("li")` do?

- A) Finds an existing `<li>` element on the page
- B) Creates a new `<li>` element in memory
- C) Deletes an `<li>` element from the page
- D) Changes the text of an existing `<li>` element

Building a List with a Loop

Combine arrays, loops, and DOM methods:

```
let skills = ["JavaScript", "HTML", "CSS"];
let skillsList = document.querySelector("#skills ul");

for (let i = 0; i < skills.length; i++) {
  let skill = document.createElement("li");
  skill.textContent = skills[i];
  skillsList.appendChild(skill);
}
```

This is the core pattern for the skills section in your assignment.

Check for Understanding

In the loop above, what does `skillsList.appendChild(skill)` do?

- A) Creates a new `<li>` element
- B) Sets the text inside `<li>`
- C) Adds the `<li>` into the `<ul>` on the page
- D) Selects the `<ul>` from the document

Apply

Core Activity: Live Coding Demo

We'll build the assignment together, step by step.

Open your portfolio project now:

- Open `index.html` and `index.js` in your editor
- Open `index.html` in your browser

If you don't have `js/index.js` yet, we'll create it together.

Step 1 — Link JavaScript to HTML

In `index.html`, add this just before the closing `</body>` tag:

```
<script src="js/index.js"></script>
```

Then create the file `js/index.js`.

Open `index.js` and add:

```
console.log("JavaScript is connected!");
```

Save and refresh. Check the browser console. Do you see the message?

Step 2 — Add a Footer with JavaScript

In `index.html`, add `<footer></footer>` before the `<script>` tag.

Then in `index.js`:

```
let today = new Date();
let thisYear = today.getFullYear();

let footer = document.querySelector("footer");
let copyright = document.createElement("p");
copyright.innerHTML = `&copy; ${thisYear} Your Name`;
footer.appendChild(copyright);
```

Save and refresh. You should see your name and year at the bottom.

Step 3 — Add a Skills Section in HTML

Before moving to JavaScript, add this to `index.html`:

```
<section id="skills">
  <h2>Skills</h2>
  <ul></ul>
</section>
```

Place it above your "Connect" section.

Save and refresh. You should see the "Skills" heading.

Step 4 — Build the Skills List

Back in `index.js`, add:

```
let skills = ["JavaScript", "HTML", "CSS", "GitHub"];

let skillsSection = document.querySelector("#skills");
let skillsList = skillsSection.querySelector("ul");

for (let i = 0; i < skills.length; i++) {
  let skill = document.createElement("li");
  skill.textContent = skills[i];
  skillsList.appendChild(skill);
}
```

Save and refresh. Your skills should appear as a list.

Extension Activity

Only if students are ready and time allows.

Extension — Highlight a Skill on Click

Add an event listener to make skills interactive:

```
let allSkills = skillsList.querySelectorAll("li");

allSkills.forEach(function(item) {
  item.addEventListener("click", function() {
    item.style.fontWeight = "bold";
    item.style.color = "#FF5C35";
  });
});
```

Try it: Click a skill in the browser. What happens?

Assignment Preview

This week you will connect JavaScript to your portfolio:

- Create `js/index.js` and link it to your HTML with `<script>`
- **Footer:** use `new Date()` to insert copyright year and your name
- **Skills:** create an array, loop through it, and build a `<li>` for each skill
- **Style:** use flexbox or grid to lay out your skills in CSS

By the end: your HTML provides the structure, your JS fills in the content.

Tips for the Assignment

1. **Work in small steps** — add one piece, refresh, check the console
2. **Use** `console.log()` to inspect variables before using them
3. **Check your selectors** — if nothing appears, `console.log(footer)` to see if it's `null`
4. **Stuck on a method?** MDN has clear examples for every DOM method
5. **Ask for hints, not answers:**

"My querySelector is returning null. Can you give me a hint about what might be wrong?"

Wrap-Up

Zoom chat: Rate your confidence from 1 to 5

How comfortable do you feel selecting elements and adding them to the page?

Before next session:

- Complete the **lesson materials** (Odin Project or Scrimba)
- Finish the **assignment** — footer, skills list, and CSS styling
- Post questions in the **Slack** `#discussion` channel anytime

Resources

Resource	What it's for
Odin Project — DOM Manipulation	Written lesson with exercises
Odin Project — Form Basics	Background for next week
Scrimba — JavaScript Deep Dive	Video walkthroughs
MDN — Introduction to the DOM	Reference for all DOM methods
MDN — Document.querySelector	Selector syntax and examples

Next week: Async Programming and Promises!

Great work today!

Your portfolio is now powered by JavaScript.

Questions? Reach out on Slack anytime.